

NCP 1.0 System Design

Low-overhead control, exact lifecycle ownership, bounded resources, and ecosystem boundaries

NCP project design note

17 August 2026

PROPOSED B01 DESIGN • UNRELEASED 1.0.0-rc.1 • NO RELEASE AUTHORITY

Abstract

This report presents the proposed high-level architecture for the unreleased NCP 1.0.0-rc.1 candidate. It separates control, perception, action, and observation traffic. It also defines exact identity, lifecycle, retry, freshness, and resource boundaries. The selected runtime prepares immutable context once. A hot frame is bounded, authenticated, decoded once, located exactly, admitted by one short owner transition, and handed to a finite plane queue. The mathematical sections derive each symbolic bound from named variables and assumptions. They cover latency decomposition, finite memory, single-flight lifecycle ownership, exact retry, checked ordinals, queue isolation, fixed-layout work, step windows, and command freshness. These equations are design accounting. They are not measured performance or operational qualification. The report is informative B01 material. It does not accept an ADR, allocate B03 values, change the normative wire, certify security or physical safety, qualify an ecosystem peer, or authorize publication. NCP remains release-blocked until every declared gate has exact evidence.

Contents

1	Reading contract and release boundary	2
2	System at a glance	3
3	Notation, units, and proof discipline	4
4	Trust topology and ordered admission	5
5	Prepared runtime and hot-path latency	6
5.1	Stage intervals	6
5.2	Worst-case design bound	7
5.3	Allocation and copy accounting	7
6	Finite memory and queue isolation	7
6.1	Store-by-store derivation	7
6.2	Portable and local allocation domains	8
6.3	Plane queue envelope	8

7	Session lifecycle mutation owner	10
7.1	Single-flight invariant	10
7.2	Durable transition order	10
7.3	Exact retry relation	11
7.4	Checked ordinal and no-reuse proof	11
7.5	No dedicated service per session	12
8	Streams, positions, and fixed-layout work	12
8.1	Position allocation	12
8.2	Fixed-layout frame size	12
9	Simulation-step execution	13
9.1	Window and memory derivation	13
9.2	Strict execution order	14
9.3	Exact response correlation	15
10	Plant command admission and freshness	15
10.1	Receiver-owned deadline	15
10.2	Active command predicate	16
10.3	Restrictive modes	16
11	Body effect boundary	16
12	Extensions and bounded reassembly	16
12.1	Complete bounded-state envelope	18
13	Ecosystem dependency direction	19
14	Current implementation boundary	20
15	Twenty-lens design review	22
16	Evidence and verification plan	23
17	Equation audit and symbol glossary	24
18	Conclusion	26

1 Reading contract and release boundary

Repository HEAD is the unreleased and release-blocked `1.0.0-rc.1` candidate. Its wire is `1.0`. Its compact protobuf contract hash is `163acc57d8a62b66`. The latest immutable release is `v0.8.0`, which uses a different wire.

This report describes the selected target for B01 review. The complete target is not the current runtime. B02 must authorize a deliberate rebaseline. B03 must allocate exact identities and numeric limits. Dependency-ready implementation tasks must then change each normative and generated surface together.

CLAIM BOUNDARY

Local source review, equations, diagrams, PDF reproduction, tests, and semantic models cannot establish live security, independent interoperability, or physical safety. They cannot establish performance qualification, signed provenance, consumer qualification, or release readiness.

The design uses three evidence states:

1. **Selected target** means proposed architecture for B01 review.
2. **Current prototype** means source that explores only part of that target.
3. **Implemented contract** requires authorized rebaseline, allocation, generation, implementation, and exact conformance evidence.

This report presents the selected target at the high-level design boundary. It does not complete the runtime or protocol. A diagram or equation cannot promote the target to an implemented contract.

2 System at a glance

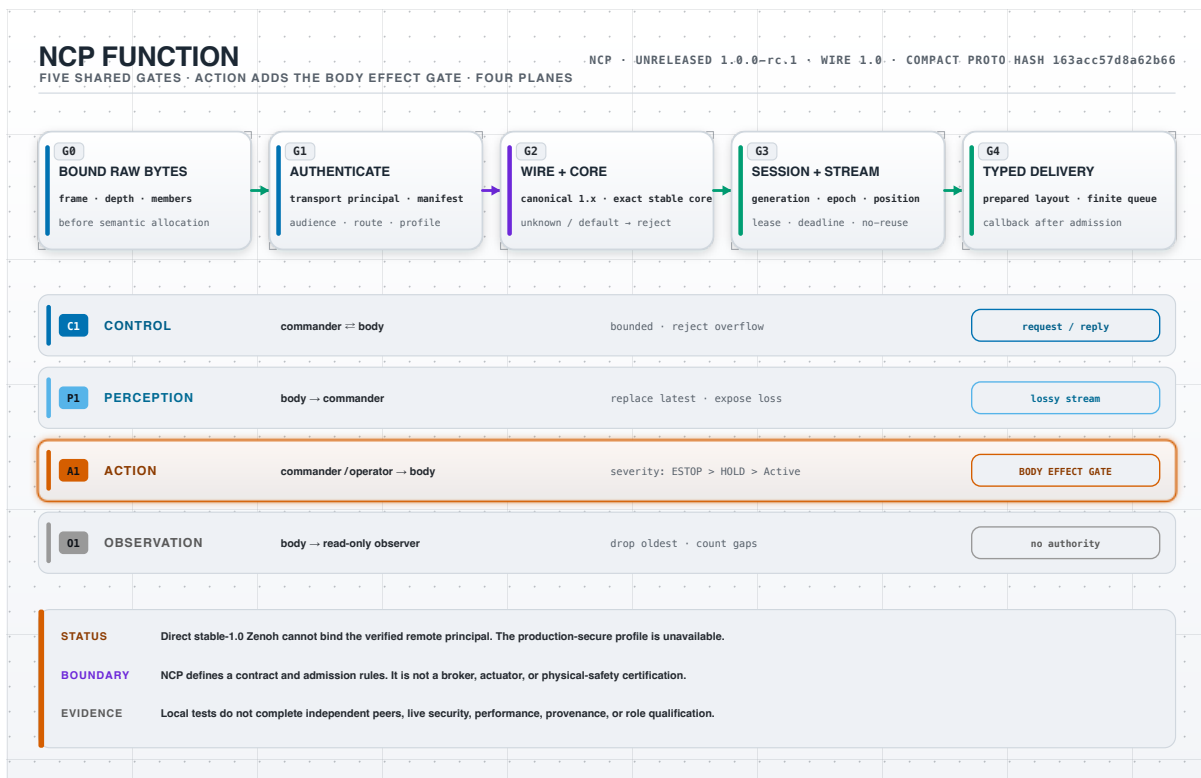


Figure 1: NCP admission and typed-delivery overview.

Figure 1 has one left-to-right reading order. A receiver first bounds raw input. It then authenticates ingress and checks the exact candidate wire identity. Session and stream state are checked before typed delivery. These are five shared gates. Action traffic evaluates a sixth, conditional body-owned effect predicate before software admission can succeed.

The four planes have different workload and overload properties:

Table 1: Plane ownership and overload policy.

Plane	Primary publisher	Ownership target	Overload result
Control	Commander or body	Request owner with exact idempotency context	Reject before effect
Perception	Body	Source stream owner	Replace latest and expose loss
Action	Commander or enrolled emergency source	Body owner and executor gate	Preserve restrictive priority
Observation	Body	Grant-bound projection owner	Drop oldest and count gaps

NCP defines contracts, admissions, libraries, and transport bindings. NCP is not a broker, actuator, or physical-safety certification.

3 Notation, units, and proof discipline

All counts are nonnegative integers. All byte quantities use bytes, not characters. All local time values use one receiver-owned monotonic clock incarnation. A digest is a content locator or binding. A digest does not replace an exact byte comparison when the design requires exact bytes.

Table 2: Core mathematical notation.

Symbol	Domain or unit	Meaning
C_x	nonnegative integer elements	Declared capacity of bounded store or queue x
B_x	nonnegative integer bytes per unit	Maximum portable charge for one capacity unit
B_0	nonnegative integer bytes	Fixed owner and store overhead
J_x	finite nonnegative local units	Profile-bound local allocator charge
T_i	nonnegative receiver duration	Duration of hot-path stage i
U_i	finite nonnegative receiver duration	Declared upper bound for stage i
o	finite nonnegative integer	Issuer operation ordinal
p	finite nonnegative integer	Stream or step position
d	nonnegative receiver monotonic tick	Exclusive deadline
$\ b\ $	nonnegative integer bytes	Exact length of byte string b

Each displayed inequality states its assumptions. Checked arithmetic rejects overflow before state mutation. A finite expression is not operational until B03 selects every input bound.

4 Trust topology and ordered admission

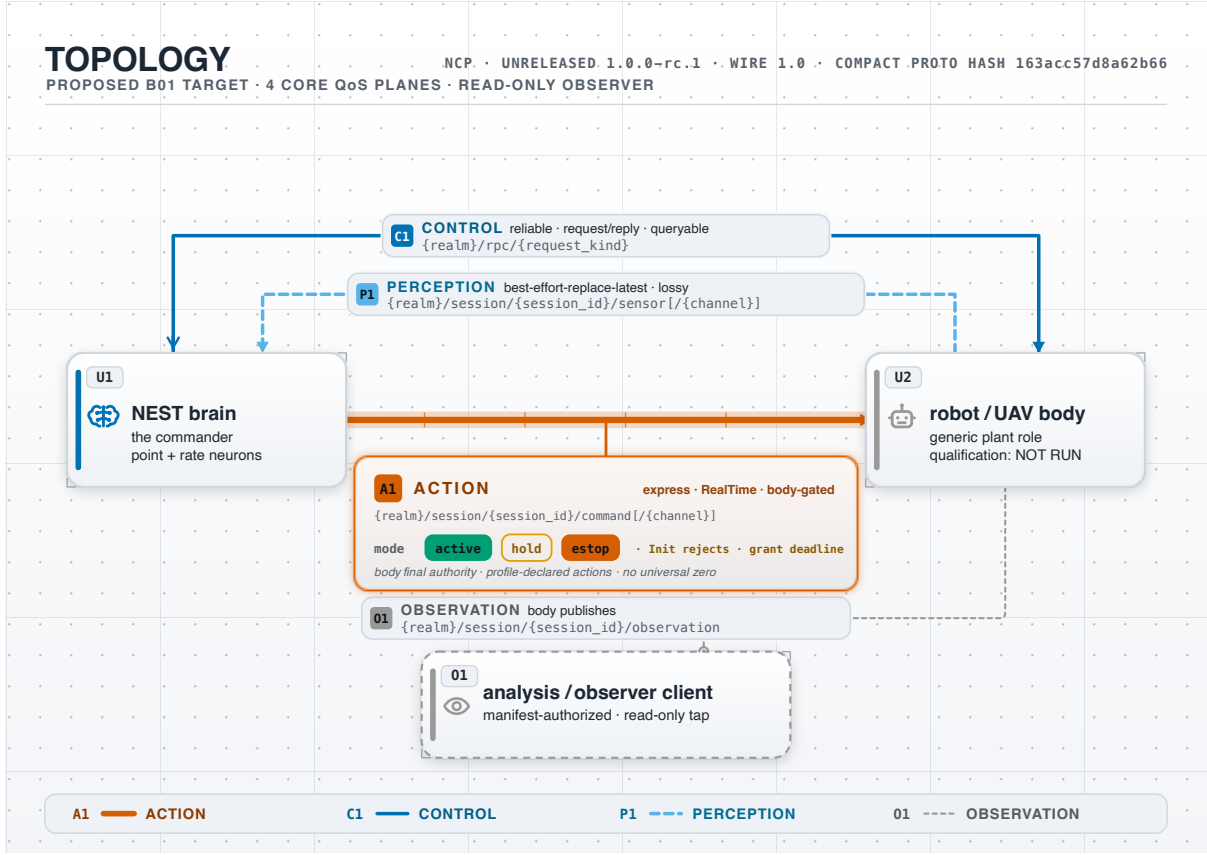


Figure 2: NCP commander-body topology and trust boundary.

The topology figure separates direct authenticated ingress from explicit forwarding. A direct peer must bind the verified transport principal. A forwarder terminates one trust context and creates another authenticated context. Copied identity bytes are not transport evidence.

Let f be one received frame. Let $q_j(f)$ be the Boolean result of admission gate j . The ordered gate set is

$$\mathcal{G} = (q_0, q_1, q_2, q_3, q_4, q_5) = (q_{\text{raw}}, q_{\text{principal}}, q_{\text{wire}}, q_{\text{session}}, q_{\text{typed}}, q_{\text{effect}}). \quad (1)$$

For a non-action frame, $q_{\text{effect}}(f) = \text{true}$ by definition. For an action frame, that predicate means body-owned software admission at the effect boundary. The complete admission predicate is the conjunction

$$A(f) = \bigwedge_{j=0}^5 q_j(f). \quad (2)$$

The implementation does not evaluate equation (2) as an unordered set. It evaluates the prefix predicates

$$A_{-1}(f) = \text{true}, \quad (3)$$

$$A_j(f) = A_{j-1}(f) \wedge q_j(f), \quad 0 \leq j \leq 5. \quad (4)$$

Gate $j + 1$ can run only when $A_j(f)$ is true. Therefore raw bounds precede identity-sensitive diagnostics. The first false gate supplies the closed rejection class.

Derivation 4.1 (First-failure order). If at least one gate is false, let $k = \min\{j \mid q_j(f) = \text{false}\}$. Equation (4) gives $A_{k-1}(f) = \text{true}$ and $A_k(f) = \text{false}$. No q_j for $j > k$ is evaluated. The receiver therefore emits no diagnostic that depends on an unauthenticated or unbounded later field.

The effect gate applies only to action. Other planes terminate at typed delivery and their finite queue. The body remains the final software authority before an actuator boundary.

5 Prepared runtime and hot-path latency

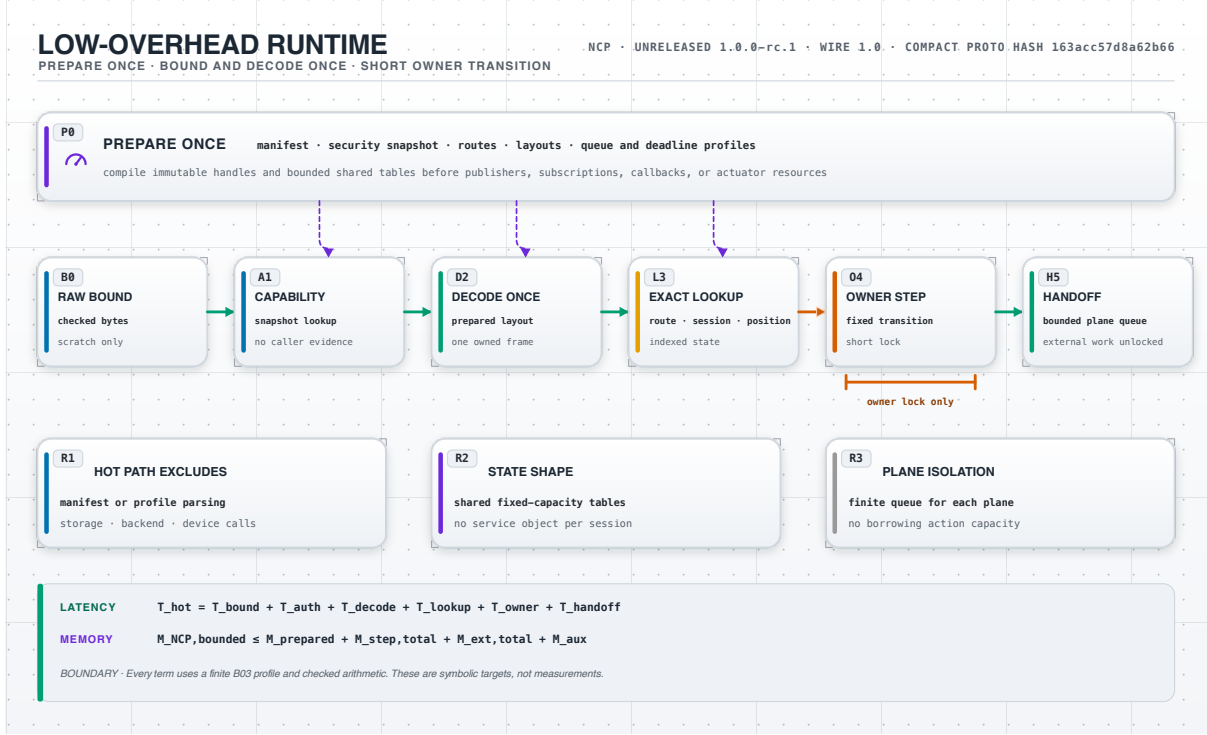


Figure 3: Prepared NCP runtime and symbolic cost model.

Preparation validates deployment input and compiles immutable handles. It reserves shared state before publishers, subscriptions, callbacks, or actuator resources are exposed. The repeated frame path then contains six local stages.

5.1 Stage intervals

Let the ordered stages be

$$\mathcal{I} = (\text{bound}, \text{auth}, \text{decode}, \text{lookup}, \text{owner}, \text{handoff}). \quad (5)$$

For each stage $i \in \mathcal{I}$, let t_i^{in} and t_i^{out} be receiver monotonic ticks from one clock incarnation. A cross-incarnation pair is invalid. Define the local duration

$$T_i = t_i^{\text{out}} - t_i^{\text{in}} \geq 0. \quad (6)$$

The serial accounting model assumes

$$t_i^{\text{out}} = t_{i+1}^{\text{in}} \quad (7)$$

for adjacent stages. The equality is an accounting boundary. It does not require a thread or process boundary.

Starting with the first input tick and ending at handoff gives

$$T_{\text{hot}} = t_{\text{handoff}}^{\text{out}} - t_{\text{bound}}^{\text{in}} \quad (8)$$

$$= \sum_{i \in \mathcal{I}} (t_i^{\text{out}} - t_i^{\text{in}}) \quad (9)$$

$$= T_{\text{bound}} + T_{\text{auth}} + T_{\text{decode}} + T_{\text{lookup}} + T_{\text{owner}} + T_{\text{handoff}}. \quad (10)$$

Equation (9) follows by inserting the adjacent boundary ticks from equation (7). Every internal tick occurs once with each sign, so the internal terms cancel.

5.2 Worst-case design bound

Suppose the selected profile supplies a finite stage ceiling U_i such that

$$0 \leq T_i \leq U_i \quad \text{for each } i \in \mathcal{I}. \quad (11)$$

Monotonicity of addition gives

$$T_{\text{hot}} = \sum_{i \in \mathcal{I}} T_i \leq \sum_{i \in \mathcal{I}} U_i = U_{\text{hot}}. \quad (12)$$

This is a conditional design bound. B03 must select each U_i . Later measurement must show that the implementation satisfies those ceilings on the qualified platform.

Transport delay precedes $t_{\text{bound}}^{\text{in}}$. Queued work, application work, and device work follow $t_{\text{handoff}}^{\text{out}}$. Those durations are outside T_{hot} .

5.3 Allocation and copy accounting

Let $a_i, c_i \in \mathbb{Z}_{\geq 0}$. The value a_i is the number of heap allocations caused by stage i after preparation. The value c_i is the number of full-payload copies. The totals are

$$A_{\text{hot}} = \sum_{i \in \mathcal{I}} a_i, \quad (13)$$

$$C_{\text{hot}} = \sum_{i \in \mathcal{I}} c_i. \quad (14)$$

For a fixed-layout compact frame with a caller-owned contiguous receive buffer, the target is

$$A_{\text{hot}} = 0, \quad C_{\text{hot}} = 0 \quad (15)$$

between ingress and typed handoff. A transport can still own one bounded receive buffer. A segmented transport can make one bounded flattening copy. If preparation reserved that buffer, then $A_{\text{hot}} = 0$ and $C_{\text{hot}} \leq 1$. An on-demand flattening buffer also adds one allocation and is outside the zero-allocation target.

Equation (15) is a target, not current evidence. Compatibility JSON can allocate within declared raw and semantic budgets.

6 Finite memory and queue isolation

6.1 Store-by-store derivation

Let $\mathcal{S} = \{s, c, r, w\}$ identify active session slots, prepared contexts, retained terminal results, and optional waiters. Store x has a profile capacity C_x . One capacity unit has a maximum charged size B_x bytes.

For every $x \in \mathcal{S}$, B03 selects $C_x, B_x \in \mathbb{Z}_{\geq 0}$. It also selects $B_0 \in \mathbb{Z}_{\geq 0}$. These are declared reservation charges, not sampled heap usage.

The portable charge B_x includes each specified variable component that scales with capacity. This includes content, backing storage, index entries, and metadata. It excludes runtime-specific allocator overhead, which belongs to the separate J_x domain. Let B_0 cover fixed portable owner and store overhead that does not scale with a declared capacity.

If store x reserves space for C_x units, then

$$M_x \leq C_x B_x. \quad (16)$$

The product has units

$$[C_x B_x] = [\text{elements}] [\text{bytes/element}] = [\text{bytes}]. \quad (17)$$

The stores are disjoint accounting domains. Adding equation (16) across the four domains gives

$$M_{\text{lifecycle}} \leq B_0 + \sum_{x \in \mathcal{S}} M_x \quad (18)$$

$$\leq B_0 + \sum_{x \in \mathcal{S}} C_x B_x \quad (19)$$

$$= B_0 + C_s B_s + C_c B_c + C_r B_r + C_w B_w. \quad (20)$$

If waiters are disabled, $C_w = 0$. The bound still includes the fixed overhead B_0 . This avoids the false conclusion that an empty preallocated table has zero memory.

All products and sums use checked arithmetic. Choose one fixed ordering $(x_1, \dots, x_{|\mathcal{S}|})$ of $\mathcal{S}$. For $1 \leq k \leq |\mathcal{S}|$, define the partial sum

$$P_k = B_0 + \sum_{j=1}^k C_{x_j} B_{x_j}. \quad (21)$$

The receiver computes each product first. It then computes P_k in a fixed order. Any multiplication or addition overflow rejects the profile before allocation.

6.2 Portable and local allocation domains

Portable bytes do not prove a physical heap bound for every language runtime. Let $J_x(C_x)$ be the local allocator charge selected for store x . All J_x values and J_0 use one runtime-profile accounting unit. A local implementation independently enforces

$$J_{\text{local}} \leq J_0 + \sum_{x \in \mathcal{S}} J_x(C_x). \quad (22)$$

The receiver admits a profile only when both the portable byte bound and the local charge bound pass. The function J_x must be finite and monotone over the permitted capacity range. It can include container nodes, capacity slack, handles, and allocator metadata. A JavaScript implementation must describe J_{local} as an abstract charge unless it controls the complete typed-array representation.

6.3 Plane queue envelope

Let $\mathcal{D}_{\text{plane}} = \{\text{ctl}, \text{per}, \text{act}, \text{obs}, \text{ext}\}$ identify control, perception, action, observation, and extension queues. Queue p has count capacity Q_p and aggregate byte capacity D_p . Let n_p be its retained-item count.

For every $p \in \mathcal{D}_{\text{plane}}$, B03 selects $Q_p \in \mathbb{Z}_{>0}$ and $D_p, B_{p,\text{meta}}, G_p \in \mathbb{Z}_{\geq 0}$. The value G_p is the largest representable visible-loss count. The current loss count satisfies $0 \leq g_p \leq G_p$. The live item count n_p and every byte-string length are nonnegative integers. The metadata charge covers the complete portable per-item metadata. Define the retained byte charge

$$S_p = \sum_{k=1}^{n_p} \|b_{p,k}\|. \quad (23)$$

The current queue invariant is $0 \leq n_p \leq Q_p$ and $0 \leq S_p \leq D_p$. The plane policy examines a new item b without retaining it. It either rejects or selects one deterministic allowable victim set $E_p(b) \subseteq \{1, \dots, n_p\}$. Define

$$e_p(b) = |E_p(b)|, \quad R_p(b) = \sum_{k \in E_p(b)} \|b_{p,k}\|. \quad (24)$$

Control and extension select the empty set under reject-on-overflow policy. Perception can select the current replaceable latest item. Observation selects the oldest items in order. Action can select only work that its restrictive severity policy permits it to displace. If no allowable victim set makes room, the incoming item rejects and the queue does not change.

Before mutation, the owner computes the survivor state

$$n_p^- = n_p - e_p(b), \quad S_p^- = S_p - R_p(b). \quad (25)$$

The subset relation gives $0 \leq e_p(b) \leq n_p$ and $0 \leq R_p(b) \leq S_p$. The subtraction and every later sum use checked arithmetic. Admission requires

$$0 \leq n_p^- < Q_p, \quad (26)$$

$$0 \leq S_p^- \quad \wedge \quad S_p^- + \|b\| \leq D_p. \quad (27)$$

Before mutation, the owner also computes

$$g_p' = \text{checked_add}(g_p, e_p(b)) \leq G_p. \quad (28)$$

If the checked successor is unavailable, the owner seals or retires the affected stream and leaves the queue unchanged.

One atomic owner transition reserves the incoming item, records the visible loss, removes the selected victims, and installs the new item. The resulting state is

$$n_p' = n_p^- + 1 \leq Q_p, \quad (29)$$

$$S_p' = S_p^- + \|b\| \leq D_p. \quad (30)$$

A count limit alone is insufficient because item size varies. The queue reserves both one count unit and the complete byte charge before it copies or retains the incoming item.

Let $B_{\text{queue},0} \in \mathbb{Z}_{\geq 0}$ be the fixed portable charge for queue owners and indexes that does not scale with a declared queue capacity. The total portable queue envelope is

$$M_{\text{queues}} \leq B_{\text{queue},0} + \sum_{p \in \mathcal{D}_{\text{plane}}} (D_p + Q_p B_{p,\text{meta}}). \quad (31)$$

The lifecycle stores and plane queues are disjoint portable accounting domains. Combining equations (19) and (31) gives

$$M_{\text{prepared}} \leq B_0 + \sum_{x \in \mathcal{S}} C_x B_x + B_{\text{queue},0} + \sum_{p \in \mathcal{D}_{\text{plane}}} (D_p + Q_p B_{p,\text{meta}}). \quad (32)$$

Let $J_{\text{queue},0}$ be the fixed local-runtime charge for the queue owners. Let $J_p^{\text{queue}}(Q_p, D_p)$ be queue p 's finite local-runtime charge. It is monotone in both arguments and uses the same runtime-profile accounting unit as J_0 . The independently checked local queue bound is

$$J_{\text{queues}} \leq J_{\text{queue},0} + \sum_{p \in \mathcal{D}_{\text{plane}}} J_p^{\text{queue}}(Q_p, D_p). \quad (33)$$

The lifecycle stores and plane queues are disjoint accounting domains. Their combined local charge therefore satisfies

$$J_{\text{prepared}} = J_{\text{local}} + J_{\text{queues}} \quad (34)$$

$$\leq J_0 + \sum_{x \in \mathcal{S}} J_x(C_x) + J_{\text{queue},0} + \sum_{p \in \mathcal{D}_{\text{plane}}} J_p^{\text{queue}}(Q_p, D_p). \quad (35)$$

No queue can borrow another plane's D_p , Q_p , or local charge. Therefore observation pressure cannot consume reserved action capacity.

Limitation 6.1 (Meaning of the memory bounds). Equations (20), (22), and (35) exclude transport buffers and unrelated process memory. They also exclude device memory. They become operational only after B03 selects exact numeric profiles.

7 Session lifecycle mutation owner

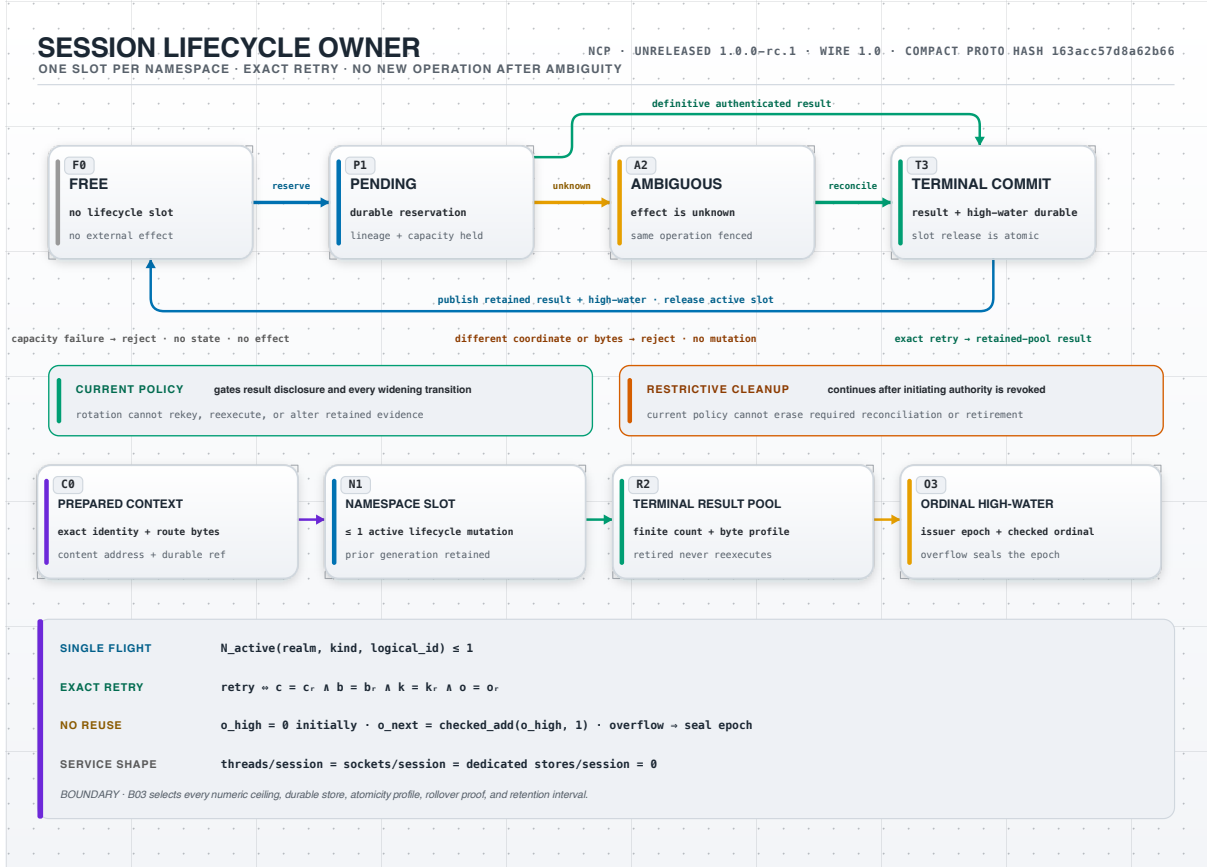


Figure 4: Single-flight session lifecycle owner and exact retry model.

One bounded slot serializes open, replacement, and close for one namespace. The namespace is the tuple

$$\nu = (\text{realm}, \text{session_kind}, \text{logical_id}). \quad (36)$$

The active states are **PENDING** and **AMBIGUOUS**. A terminal record lives in a separate retained-result pool.

7.1 Single-flight invariant

Let $\mathcal{O}_\nu$ be all lifecycle records for namespace ν . Define

$$N_{\text{active}}(\nu) = \sum_{u \in \mathcal{O}_\nu} \mathbf{1}[\text{state}(u) \in \{\text{PENDING}, \text{AMBIGUOUS}\}]. \quad (37)$$

The owner has one active slot for ν . Each transition into an active state first proves that the slot is free. Therefore

$$0 \leq N_{\text{active}}(\nu) \leq 1. \quad (38)$$

The upper bound is structural. A mutex alone is insufficient across processes. A shared writer must also bind one fenced writer epoch.

7.2 Durable transition order

Let L be the prior lineage. Let E be the external effect state. Let R be the reserved terminal-result cell. Let K be the prepared-context reference. The transition from free to pending is ordered as

$$(\text{FREE}, L) \xrightarrow{\text{reserve}(R, K, L)} (\text{PENDING}, L, R, K) \xrightarrow{\text{external work}} E. \quad (39)$$

The first edge is crash-consistent. External work cannot start before that edge is durable. Capacity failure therefore leaves no partial slot and starts no external effect.

An unknown result does not create a new operation:

$$(\text{PENDING}, c) \xrightarrow{\text{unknown effect}} (\text{AMBIGUOUS}, c). \quad (40)$$

The immutable coordinate c remains unchanged. Reconciliation queries the same backend coordinate and then moves the same record to terminal commit.

Terminal commit has the order

$$\text{publish}(r, o_{\text{high}}) \prec \text{release}(\text{active slot}), \quad (41)$$

where $\prec$ means *commits before*. A later operation cannot observe a free slot without also observing the retained result and advanced high-water.

7.3 Exact retry relation

Let the retained operation be

$$u_r = (c_r, b_r, k_r, o_r). \quad (42)$$

Here c_r is the immutable coordinate. The value b_r is the exact request bytes. The value k_r is the exact admission-context bytes. The value o_r is the issuer ordinal.

For retry input $u = (c, b, k, o)$, define

$$R_{\text{exact}}(u, u_r) \iff (c = c_r) \wedge (b = b_r) \wedge (k = k_r) \wedge (o = o_r). \quad (43)$$

$$R_{\text{conflict}}(u, u_r) \iff \neg R_{\text{exact}}(u, u_r). \quad (44)$$

Equality of b and k is byte-for-byte equality. A digest can select a candidate record. The receiver still exact-compares the retained bytes before it returns or mutates state.

The relation in equation (43) is reflexive, symmetric, and transitive because each component uses exact equality. Therefore it partitions retry inputs into equivalence classes. Only the retained class can observe the retained result.

7.4 Checked ordinal and no-reuse proof

Let $O_{\text{max}} \in \mathbb{Z}_{>0}$ be finite for one issuer epoch. Define the assignable ordinal domain and initial high-water sentinel by

$$\mathcal{D}_o = \{1, 2, \dots, O_{\text{max}}\}, \quad o_{\text{high}} \in \{0\} \cup \mathcal{D}_o. \quad (45)$$

The value $o_{\text{high}} = 0$ means that the epoch has committed no operation ordinal. Otherwise, o_{high} is the largest committed ordinal. For a terminal high-water o_{high} , the successor exists only when

$$o_{\text{high}} < O_{\text{max}}. \quad (46)$$

Under that precondition, the next ordinal is

$$o_{\text{next}} = \text{checked_add}(o_{\text{high}}, 1). \quad (47)$$

Checked addition gives

$$o_{\text{next}} = o_{\text{high}} + 1 > o_{\text{high}}, \quad o_{\text{next}} \in \mathcal{D}_o. \quad (48)$$

If equation (46) is false, the epoch seals. The ordinal never wraps or saturates. An input $o \leq o_{\text{high}}$ returns retained, retired, or unavailable. It never executes again.

7.5 No dedicated service per session

Shared fixed-capacity tables serve many namespaces. The design target is

$$N_{\text{thread/session}} = N_{\text{socket/session}} = N_{\text{dedicated store/session}} = 0. \quad (49)$$

Equation (49) does not remove session data. The data lives in bounded shared owners and stores. The lifecycle memory bound remains equation (20).

8 Streams, positions, and fixed-layout work

An accepted stream declaration compiles one immutable context. The context binds route, publisher incarnation, epoch, frame class, encoding, layout, position range, replay profile, and overload policy.

8.1 Position allocation

Let $P_{\max} \in \mathbb{Z}_{>0}$ be the selected maximum position for one stream epoch. Define the assignable domain and its initial high-water sentinel by

$$\mathcal{D}_p = \{1, 2, \dots, P_{\max}\}, \quad p_{\text{high}} \in \{0\} \cup \mathcal{D}_p. \quad (50)$$

The value $p_{\text{high}} = 0$ means that the epoch has assigned no position. Otherwise, p_{high} is the largest assigned position. A successor can exist only when

$$p_{\text{high}} < P_{\max}. \quad (51)$$

Under that precondition, assignment uses

$$p_{\text{candidate}} = \text{checked_add}(p_{\text{high}}, 1). \quad (52)$$

Checked addition and equation (51) give

$$p_{\text{candidate}} = p_{\text{high}} + 1 > p_{\text{high}}, \quad p_{\text{candidate}} \in \mathcal{D}_p. \quad (53)$$

If equation (51) is false, the owner seals the stream epoch. It does not wrap, saturate, or allocate a position in a new epoch implicitly. The allocator commits the candidate before later local encode, queue, or transport work. A later failure creates a visible gap. The allocator never reuses the candidate.

Let $\mathcal{P}_{\text{assigned}}$ be the set of assigned positions and $\mathcal{P}_{\text{published}}$ the set of published positions. Then

$$\mathcal{P}_{\text{published}} \subseteq \mathcal{P}_{\text{assigned}}. \quad (54)$$

The visible gap set is

$$\mathcal{P}_{\text{gap}} = \mathcal{P}_{\text{assigned}} \setminus \mathcal{P}_{\text{published}}. \quad (55)$$

Because assignment is monotone, no element of $\mathcal{P}_{\text{gap}}$ can later identify a different payload.

8.2 Fixed-layout frame size

Let $n_v, N_v, H_f \in \mathbb{Z}_{\geq 0}$ and $w_v \in \mathbb{Z}_{>0}$. A prepared frame contains n_v numeric values, each encoded value uses w_v bytes, and the fixed header uses H_f bytes. The complete frame length is

$$L_{\text{frame}} = H_f + n_v w_v. \quad (56)$$

Suppose the profile selects $0 \leq n_v \leq N_v$. Then

$$L_{\text{frame}} = H_f + n_v w_v \quad (57)$$

$$\leq H_f + N_v w_v = L_{\text{frame}}^{\max}. \quad (58)$$

The receiver checks n_v and the complete length before it decodes values. The names, units, ranges, and offsets live in the prepared layout. The product and sum use checked arithmetic. An unavailable product or sum rejects before value decoding. Names and units are not repeated or allocated per frame.

For a direct linear decode, the work count satisfies

$$W_{\text{codec}}(n_v) = \alpha + \beta n_v, \quad (59)$$

where $\alpha, \beta \in \mathbb{Z}_{\geq 0}$ are fixed operation counts for header work and work per value. This equation is a structural operation count. It is not a time measurement.

9 Simulation-step execution

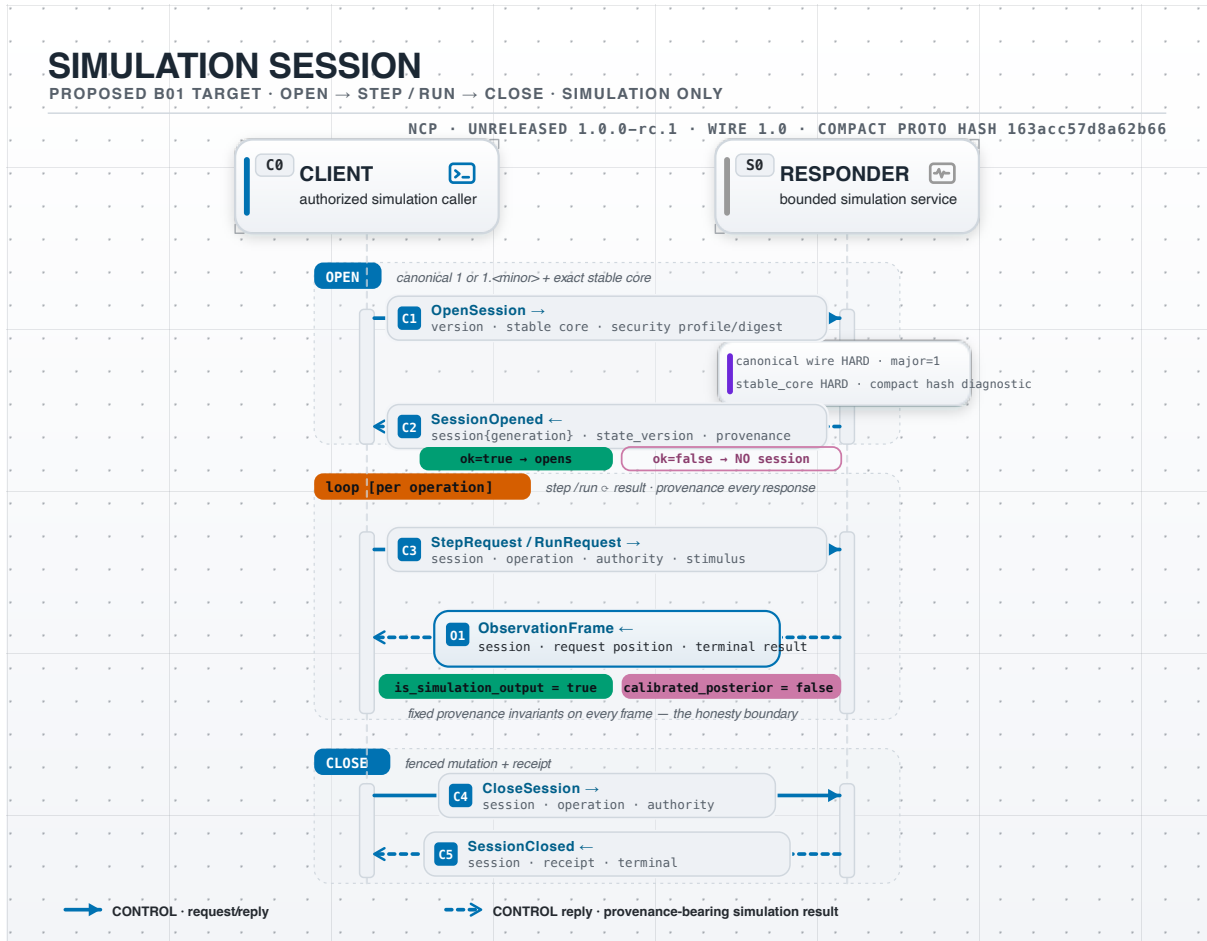


Figure 5: Simulation-step request, execution, and retained-response sequence.

Lifecycle control stays on bounded canonical JSON. Repeated simulation steps use one prepared fixed-layout request and response pair. The pair binds a live simulation generation, authority grant, exact request position, advance interval, input layout, output layout, and retained result.

9.1 Window and memory derivation

Let $C_{\text{step}} \in \mathbb{Z}_{>0}$ be the deployment maximum for prepared pipeline capacity. Let $n_{\text{open}} \in \mathbb{Z}_{\geq 0}$ be the number of reserved requests that are not terminal. Admission requires

$$0 \leq n_{\text{open}} < C_{\text{step}}. \quad (60)$$

The strict upper relation leaves one free reservation for the new request. After reservation, the invariant becomes

$$0 \leq n_{\text{open}} + 1 \leq C_{\text{step}}. \quad (61)$$

Let B_{req} and B_{res} be the maximum retained request and response bytes. Let $B_{\text{step,meta}}$ be the maximum per-slot metadata bytes. Let $B_{\text{step,0}}$ be fixed window overhead. All four byte charges are nonnegative integers. These charges are deployment maxima. Every accepted step-window profile is at or below them. Pre-reserving both directions gives

$$M_{\text{step}} \leq B_{\text{step,0}} + C_{\text{step}} (B_{\text{req}} + B_{\text{res}} + B_{\text{step,meta}}). \quad (62)$$

Let $J_{\text{step,0}}$ and $J_{\text{step,slot}}$ be finite nonnegative charges in one selected local-runtime unit. The slot charge includes its request, response, index, and container representation. These charges are deployment maxima for that runtime profile. The separate local bound is

$$J_{\text{step}} \leq J_{\text{step,0}} + C_{\text{step}} J_{\text{step,slot}}. \quad (63)$$

Each active simulation session owns at most one prepared step window. Each simultaneous window is charged to one distinct active session slot. Let n_{win} be the number of simultaneous windows. The lifecycle capacity therefore gives

$$0 \leq n_{\text{win}} \leq C_s. \quad (64)$$

Let $M_{\text{step}}^{(a)}$ and $J_{\text{step}}^{(a)}$ be the portable and local charges for window a . Summing the per-window bounds and then applying the count bound gives

$$\begin{aligned} M_{\text{step,total}} &= \sum_{a=1}^{n_{\text{win}}} M_{\text{step}}^{(a)} \\ &\leq n_{\text{win}} [B_{\text{step,0}} + C_{\text{step}} (B_{\text{req}} + B_{\text{res}} + B_{\text{step,meta}})] \\ &\leq C_s [B_{\text{step,0}} + C_{\text{step}} (B_{\text{req}} + B_{\text{res}} + B_{\text{step,meta}})], \end{aligned} \quad (65)$$

$$\begin{aligned} J_{\text{step,total}} &= \sum_{a=1}^{n_{\text{win}}} J_{\text{step}}^{(a)} \\ &\leq n_{\text{win}} [J_{\text{step,0}} + C_{\text{step}} J_{\text{step,slot}}] \\ &\leq C_s [J_{\text{step,0}} + C_{\text{step}} J_{\text{step,slot}}]. \end{aligned} \quad (66)$$

Every product and sum uses checked arithmetic. The profile checks the per-window and deployment-wide bounds before it exposes the compact step path. The owner keeps the session slot charged while its step window is active or retains a response. A close can instead move a longer-lived terminal response into the separately charged retained- result pool. That transfer and window release form one atomic owner transition. The response cell is reserved before execution. Backend work therefore cannot complete into an unbounded or unavailable result path.

9.2 Strict execution order

Let $e, p \in \mathbb{Z}_{\geq 0}$ be positions in one finite generation domain. The value e is the next executable position. A request at position p can start only when

$$p = e. \quad (67)$$

On definitive completion, the owner computes

$$e' = \text{checked_add}(e, 1). \quad (68)$$

If the successor cannot be represented, the owner seals the generation and admits no later position. An unresolved backend call retires the generation. The owner does not infer order from arrival or from a FIFO waiter list.

9.3 Exact response correlation

Let $h_p = \text{digest}(b_p)$ index the exact request bytes at position p . The request store retains b_p until the terminal response is durable. The retained response is the tuple

$$r_p = (\text{generation}, p, h_p, \text{result}). \quad (69)$$

A response can satisfy only a waiter with the same generation, position, request digest, and exact request bytes. The digest locates the candidate. The receiver exact-compares b_p before it releases the response. A reply cannot consume the next FIFO waiter by arrival order.

10 Plant command admission and freshness

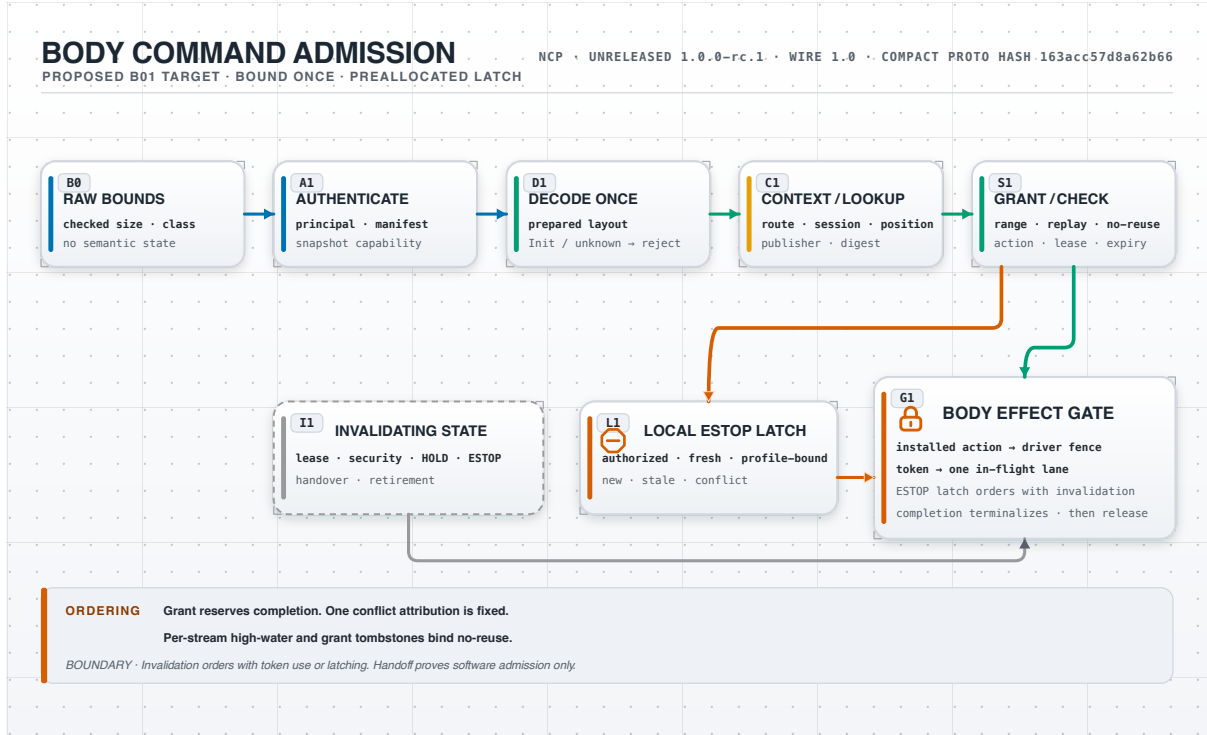


Figure 6: Body-owned plant command admission order.

The body issues live authority and command freshness. Serialized lease possession does not grant authority. Receiver arrival never starts or refreshes a remote command lifetime.

10.1 Receiver-owned deadline

Let $t_{\text{issue}} \in \mathbb{Z}_{\geq 0}$ be the receiver monotonic tick sampled when the body creates a freshness grant. Let $\Delta_{\text{max}} \in \mathbb{Z}_{>0}$ be the finite profile lifetime in ticks. Both values use one receiver clock incarnation. The exclusive deadline is

$$d = \text{checked_add}(t_{\text{issue}}, \Delta_{\text{max}}). \quad (70)$$

If checked addition overflows, the body rejects the grant before it becomes visible. The grant is live at nonnegative receiver tick t from that same clock incarnation only when

$$t_{\text{issue}} \leq t < d. \quad (71)$$

An arrival tick t_{arrival} is audit evidence. Replacing t_{issue} with t_{arrival} would create a new lifetime after network delay. That replacement is forbidden.

Let a grant cover the contiguous integer position range $[p_0, p_1]$, where $0 \leq p_0 \leq p_1$. Position admission also requires

$$p_0 \leq p \leq p_1. \quad (72)$$

The position selects the grant. The frame does not repeat the grant, lease, or lifetime.

10.2 Active command predicate

Let the Boolean predicates be:

- q_c = security snapshot and session are current,
- q_l = the exact authority lease is live,
- q_g = the freshness grant satisfies equations (71) and (72),
- q_s = stream epoch and position pass no-reuse checks,
- q_p = plant profile accepts channel, arity, units, and range,
- q_e = the bounded executor slot accepts the exact command.

An Active command is software-admitted only when

$$A_{\text{active}} = q_c \wedge q_l \wedge q_g \wedge q_s \wedge q_p \wedge q_e. \quad (73)$$

The equation stops at software admission. It does not state that a physical effect occurred.

10.3 Restrictive modes

HOLD requires an explicit known mode and a live holder. HOLD follows ordinary stream monotonicity. ESTOP can use the separately enrolled emergency lane when policy permits it. Neither mode carries a remote value vector or source coordinate. Their actions come from the installed plant profile.

Remote rejection and a body-local restrictive response are different results. A malformed or unauthorized remote request cannot claim that the remote HOLD or ESTOP effect succeeded.

11 Body effect boundary

The body owner creates one bounded executor-admission token. The token binds the exact command, plant profile, generation, authority state, freshness state, and executor domain incarnation. The actuator interface does not accept a raw map, transport message, or serialized lease.

Let a be the software-admission event and x a later physical effect. NCP can retain evidence for a . The protocol proof boundary is

$$\text{NCP} \not\models (a \implies x). \quad (74)$$

The turnstile states that NCP does not prove the implication. It is not a claim that the effect is impossible. Device, mechanical, electrical, and plant-local safety systems remain outside the protocol proof boundary. A deployment needs its own safety case and independent interlock.

12 Extensions and bounded reassembly

An extension package uses raw bounded chunks outside the stable action semantics. Let $L, L_{\max}, C \in \mathbb{Z}_{>0}$ be byte counts with $L \leq L_{\max}$. The value C is the maximum decoded chunk size. The canonical chunk count is

$$N = \text{ceil_div}(L, C) = \left\lceil \frac{L}{C} \right\rceil. \quad (75)$$

The count therefore satisfies

$$1 \leq N \leq \left\lceil \frac{L_{\max}}{C} \right\rceil = N_{\max}. \quad (76)$$

Because $C > 0$ and $N = \lceil L/C \rceil$, the defining ceiling relation is equivalent to

$$(N - 1)C < L \leq NC. \quad (77)$$

For $i \in \{0, \dots, N - 1\}$, the receiver computes the chunk offset with checked multiplication:

$$o_i = iC. \quad (78)$$

The canonical decoded length ℓ_i must satisfy

$$0 \leq o_i < L, \quad \ell_i = \min(C, L - o_i) > 0. \quad (79)$$

Treat byte ranges as half-open integer intervals. The exact destination interval is

$$I_i = [o_i, o_i + \ell_i) = [iC, \min((i + 1)C, L)). \quad (80)$$

For $i < N - 1$, equation (77) gives $(i + 1)C < L$. Those chunks have length C . The final chunk ends at L . Hence

$$I_i = \begin{cases} [iC, (i + 1)C), & 0 \leq i < N - 1, \\ [(N - 1)C, L), & i = N - 1. \end{cases} \quad (81)$$

The right endpoint of each non-final interval equals the next interval's left endpoint. The first left endpoint is zero, and the final right endpoint is L . The canonical count and lengths make these intervals a partition:

$$I_i \cap I_j = \emptyset \quad (i \neq j), \quad \bigcup_{i=0}^{N-1} I_i = [0, L). \quad (82)$$

Thus, complete index coverage supplies every object byte exactly once. It does not prove the object digest or inner schema.

Let $C_{\text{ext}} \in \mathbb{Z}_{\geq 0}$ be the deployment maximum for all active and retained outer-package slots in one activation. Let $B_{\text{ext},0} \in \mathbb{Z}_{\geq 0}$ be the deployment maximum for its fixed portable charge. Let $H_{\text{ext}}^{\max} \in \mathbb{Z}_{\geq 0}$ be the deployment maximum of the greater active or terminal portable metadata charge for one slot. Every accepted activation profile is at or below these maxima. The outer portable reservation satisfies

$$M_{\text{ext},\text{outer}} \leq B_{\text{ext},0} + C_{\text{ext}} (L_{\max} + H_{\text{ext}}^{\max}). \quad (83)$$

Each installed extension activation consumes one distinct prepared-context unit. Let n_{ext} be the number of simultaneous extension activations. Equation (20) gives

$$0 \leq n_{\text{ext}} \leq C_c. \quad (84)$$

Let $M_{\text{ext},\text{outer}}^{(a)}$ be activation a 's portable outer-package charge. Summing the per-activation bound gives

$$\begin{aligned} M_{\text{ext},\text{total}} &= \sum_{a=1}^{n_{\text{ext}}} M_{\text{ext},\text{outer}}^{(a)} \\ &\leq n_{\text{ext}} [B_{\text{ext},0} + C_{\text{ext}} (L_{\max} + H_{\text{ext}}^{\max})] \\ &\leq C_c [B_{\text{ext},0} + C_{\text{ext}} (L_{\max} + H_{\text{ext}}^{\max})]. \end{aligned} \quad (85)$$

Let $J_{\text{ext},0}$ be the fixed per-activation local charge. Let $J_{\text{ext},\text{slot}}(L_{\max}, N_{\max})$ be a finite nonnegative slot charge in one selected local-runtime unit. The slot function is monotone in both arguments. The independent local bound is

$$J_{\text{ext},\text{outer}} \leq J_{\text{ext},0} + C_{\text{ext}} J_{\text{ext},\text{slot}}(L_{\max}, N_{\max}). \quad (86)$$

Let $J_{\text{ext,outer}}^{(a)}$ be activation a 's local outer-package charge. The same count bound gives

$$\begin{aligned} J_{\text{ext,total}} &= \sum_{a=1}^{n_{\text{ext}}} J_{\text{ext,outer}}^{(a)} \\ &\leq n_{\text{ext}} [J_{\text{ext},0} + C_{\text{ext}} J_{\text{ext,slot}}(L_{\text{max}}, N_{\text{max}})] \\ &\leq C_c [J_{\text{ext},0} + C_{\text{ext}} J_{\text{ext,slot}}(L_{\text{max}}, N_{\text{max}})]. \end{aligned} \quad (87)$$

Every product, sum, and offset uses checked arithmetic. Overflow rejects the profile or activation before it allocates a prepared context or package slot. An activation context remains charged while any active package slot or terminal tombstone refers to it. The owner cannot release that context first. Therefore a retained tombstone cannot escape the C_c multiplier.

The receiver reserves the complete object and metadata budget before it retains the first chunk. It copies a new decoded chunk once into the checked range $[o_i, o_i + \ell_i)$. Duplicate bytes are idempotent. Conflicting bytes terminalize the same slot. Inner schema parsing starts only after complete object-digest verification and a second schema-specific budget reservation. The outer bounds exclude transport scratch, the inner parser arena, callback state, and unrelated process memory. B03 must select all finite inputs.

12.1 Complete bounded-state envelope

Let $\mathcal{D}_{\text{aux}} = \{\text{rx}, \text{json}, \text{extparse}, \text{callback}\}$ identify transport receive or flattening scratch, structured-JSON arenas, extension inner-parser arenas, and typed callback buffers. B03 selects one deployment-wide portable ceiling $B_y \in \mathbb{Z}_{\geq 0}$ and one finite monotone local charge $J_y(B_y)$ for every $y \in \mathcal{D}_{\text{aux}}$. It also selects fixed charges $B_{\text{aux},0}$ and $J_{\text{aux},0}$. The auxiliary envelopes are

$$M_{\text{aux}} \leq B_{\text{aux},0} + \sum_{y \in \mathcal{D}_{\text{aux}}} B_y, \quad (88)$$

$$J_{\text{aux}} \leq J_{\text{aux},0} + \sum_{y \in \mathcal{D}_{\text{aux}}} J_y(B_y). \quad (89)$$

These domains are disjoint from lifecycle, queue, step-window, and outer-package storage. Adding the previously derived deployment bounds gives the complete NCP-owned bounded-state model

$$M_{\text{NCP,bounded}} \leq M_{\text{prepared}} + M_{\text{step,total}} + M_{\text{ext,total}} + M_{\text{aux}}, \quad (90)$$

$$J_{\text{NCP,bounded}} \leq J_{\text{prepared}} + J_{\text{step,total}} + J_{\text{ext,total}} + J_{\text{aux}}. \quad (91)$$

Every component is independently finite and checked before its owner becomes reachable. No domain can borrow another domain's reservation. These equations bound declared NCP-owned state, not code, thread stacks, device memory, unrelated process state, or a universally portable physical RSS. The sum applies no alias discount. A zero-copy view is charged to its backing owner. A copied buffer is a second charge. During a cross-owner transfer, both domains remain reserved until the destination commit preserves every required recovery byte and permits source release. The sum therefore covers the transfer high-water.

13 Ecosystem dependency direction

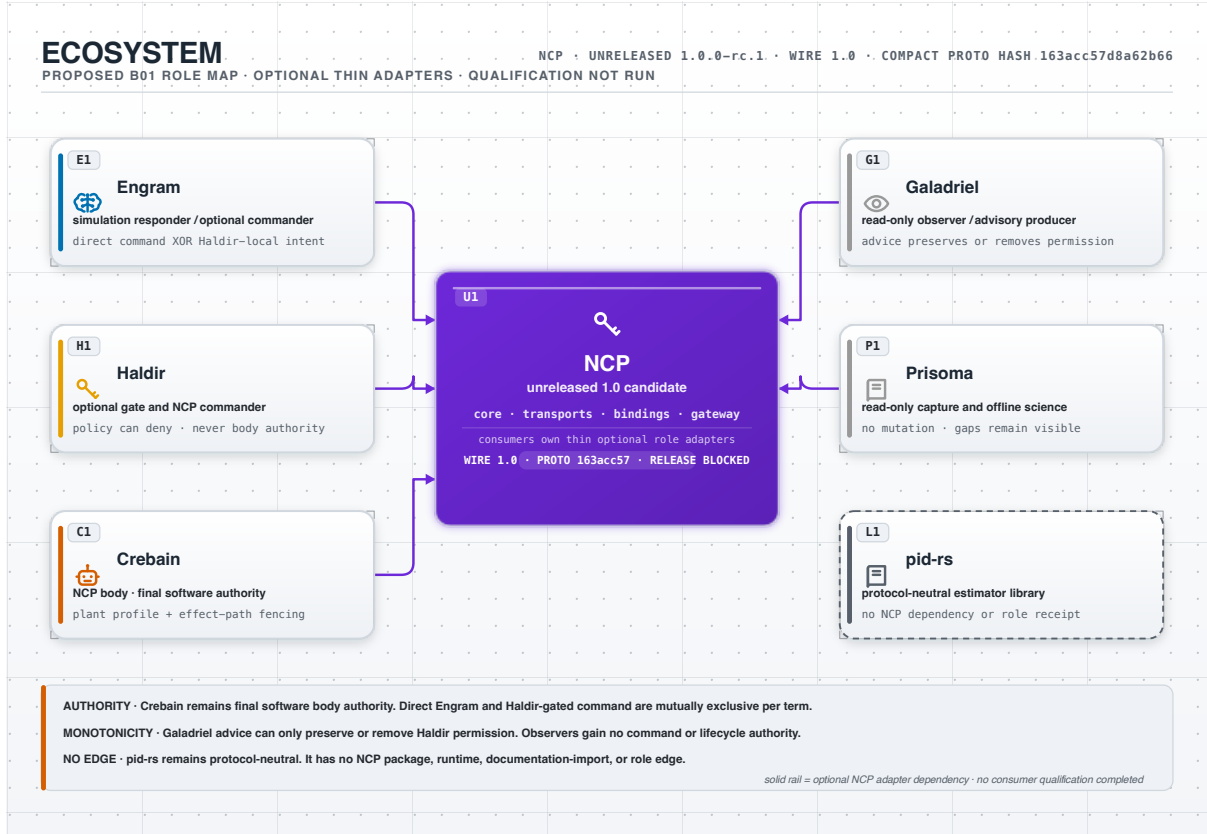


Figure 7: NCP provider and ecosystem dependency direction.

NCP owns the protocol, generated bindings, profiles, and package surfaces. Consumer projects pin NCP packages and implement thin role adapters. NCP does not import consumer application code.

Let $G = (V, E)$ be the repository dependency graph. An edge $(u, v) \in E$ means repository u depends on repository v . If $n \in V$ is NCP and $C \subset V$ is the set of consumers, the selected direction is

$$\forall c \in C : (c, n) \in E \quad \wedge \quad (n, c) \notin E. \quad (92)$$

The condition prevents a consumer-specific fork from becoming an NCP requirement.

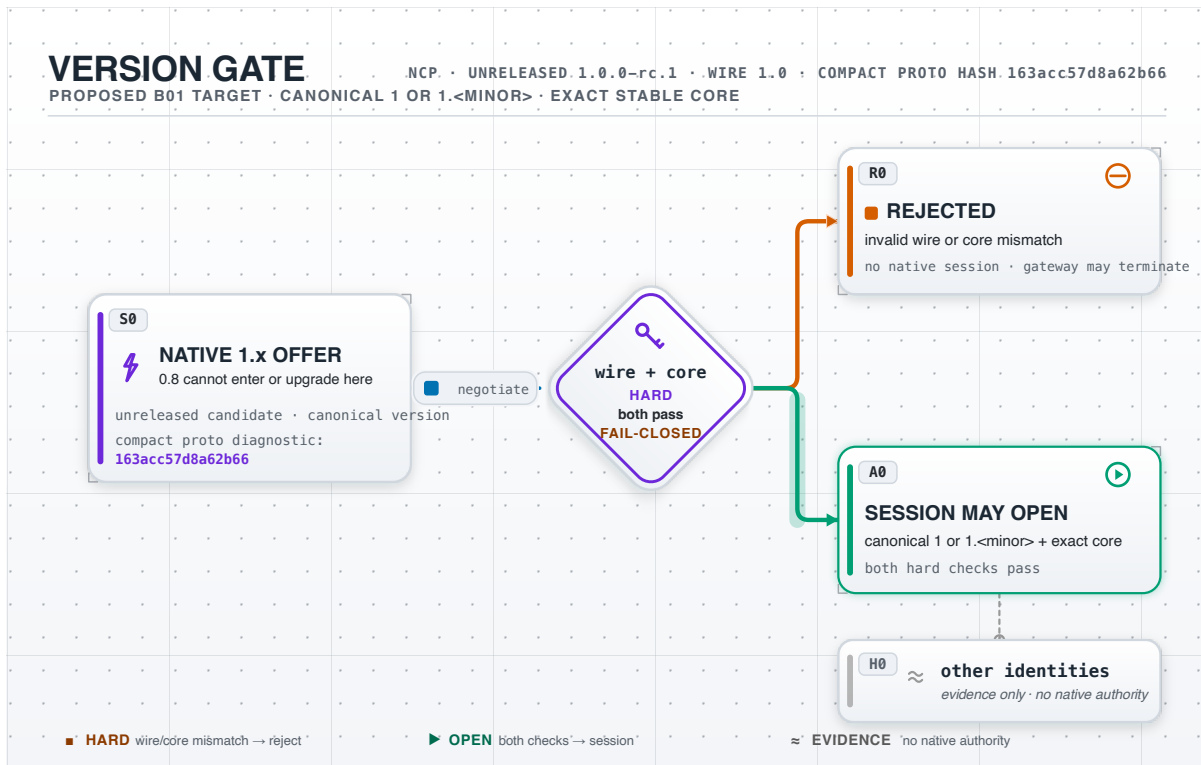


Figure 8: Candidate, stable-core, extension, and release identity domains.

The versioning figure separates wire identity from package, release, extension, and publication identity. The compact protobuf hash is not the complete normative digest. A copied protocol file does not prove migration or installed qualification.

14 Current implementation boundary

The current candidate does not implement the complete target. The most important gaps are:

- Direct **production-secure** Zenoh ingress cannot bind the verified remote principal from the callback surface.
- Callers can construct the current actor helper from supplied evidence. The helper is not an opaque receiver-minted currentness capability.
- The runtime has no one body owner that orders security cuts, leases, command streams, restrictive effects, dispositions, and executor admission.
- The TypeScript client and gateway can remove local generation state before a backend open or close has a definitive result.
- **NeuroControlLoop** stores a host-supplied serialized lease without a receiver-monotonic deadline capability.
- The tick API returns the same command type for local unadmitted fallback and an admitted transport command.
- A send outcome binds a transport position but not the exact payload digest and owner incarnation.
- Stable command wire still defaults a missing mode to **HOLD** and carries fields that the compact restrictive profiles forbid.

- No accepted compact data-plane encoding, complete no-reuse owner, durable generation issuer, or complete disposition journal exists.
- The TypeScript WebSocket client bounds pending request count but not aggregate bytes.

These are implementation gaps if B01 accepts the target. They are not alternate accepted protocol choices. B02 and B03 remain dependencies for wire-visible implementation.

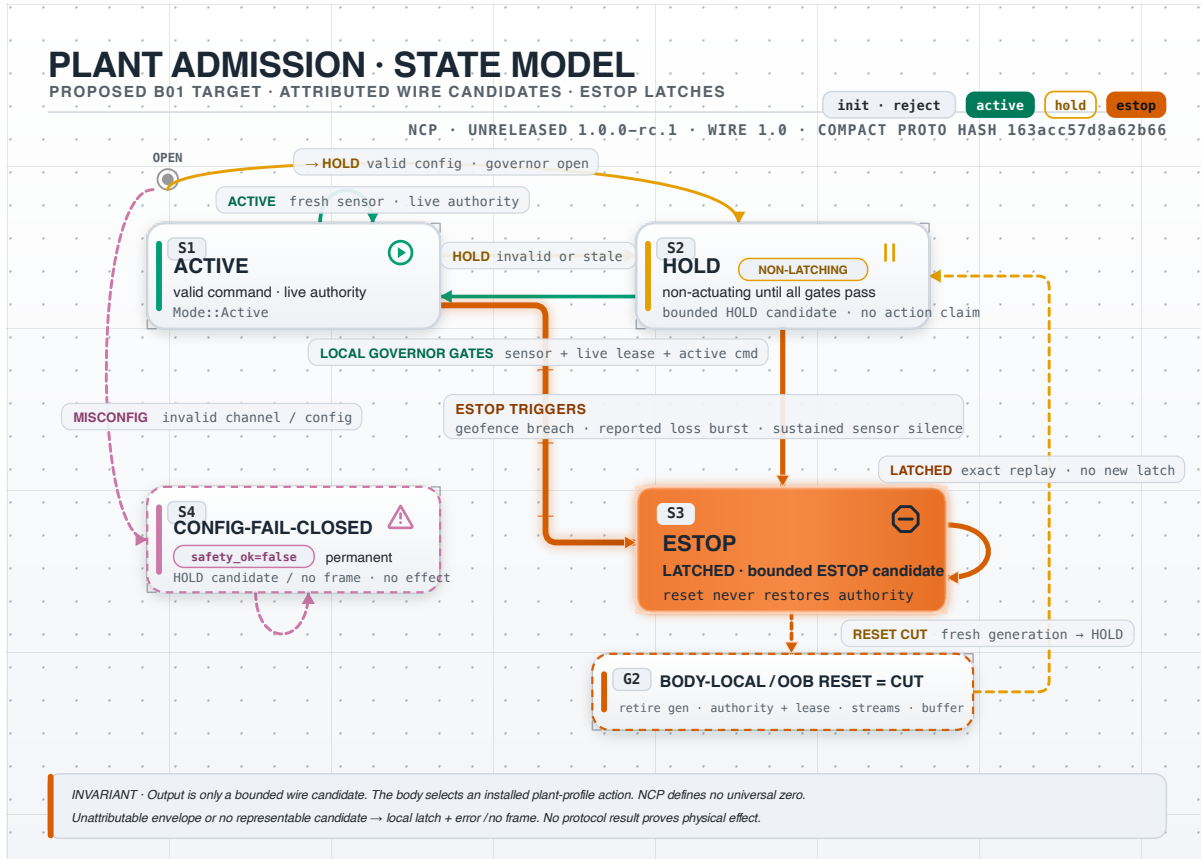


Figure 9: Current compatibility plant-admission state model.

The state model shows the candidate's normalized input, configured actions, restrictive states, and generation-cut reset. It is included to make the current compatibility boundary visible. It is not the complete proposed lifecycle, authority, stream, disposition, and executor owner. The proposed admission ordering remains the target shown earlier in this report.

15 Twenty-lens design review

This review is a maintainer-side critique. It does not satisfy the independent B01 review floor.

Table 3: Twenty-lens review of the proposed target.

Lens	Selected target	Gap or required evidence
Role and type separation	Simulation, plant, observer, extension, and local-library roles are disjoint.	B03 must allocate exact identities. Current wire still uses older shapes.
Identity and correlation	Realm, generation, stream, request, command, and package identities have explicit scopes.	Durable issuers and exact digest domains remain implementation work.
Authentication and authorization	Direct ingress and forwarded signing are default-deny and non-downgrading.	Direct verified-principal binding is unavailable. Live security campaigns are NOT RUN .
Error precedence	Raw bounds and profile selection precede identity-sensitive diagnostics.	B03 must freeze exact result codes and precedence.
Freshness and replay	Body-issued absolute grants and per-stream positions fail closed.	Exact grant encoding and digest-bound replay remain open.
Lifecycle and concurrency	One owner orders lifecycle state. Ambiguity retains the same operation.	Cross-process fencing and durable atomicity remain unimplemented.
Resources and denial of service	Every variable count composes with bytes and local allocation charges.	Numeric profiles and load evidence remain open.
Hot-path overhead	Prepared contexts remove repeated parsing. Compact frames decode once.	Compact framing and performance qualification remain NOT RUN .
Mathematical consistency	Every equation defines symbols, units, assumptions, and failure behavior.	B03 must select every finite ceiling. Implementations must match boundary tests.
Failure and recovery	Restart grants no Active authority. Unknown effects remain fenced.	Generation, no-reuse, and disposition recovery are not implemented.
Plant safety	The body owns final software admission. Physical effect is outside NCP proof.	Each deployment still needs a plant-owned safety case and interlock.
QoS and overload	Plane capacities are disjoint and finite. Restrictive work has reserved service.	Scheduler values, transport mapping, and installed load tests remain open.
SDK buffering	Pending count and aggregate bytes are both bounded.	Current TypeScript WebSocket buffering has only a count bound.
Extension isolation	Reassembly reserves raw and semantic budgets before callback.	B03 must allocate activation and parser profiles.
Evolution and compatibility	Stable-core, release, extension, and publication identities remain separate.	B02 must authorize any rebaseline.
Wire and schema parity	Each accepted object must have one source and generated forms.	Current prototype material is not normative parity.

Lens	Selected target	Gap or required evidence
Ecosystem direction	Consumers pin NCP and expose thin adapters.	No installed consumer role is qualified.
Operability	Gaps, drops, expiry, ambiguity, and insecure mode are visible.	Exact metrics and deployment procedures remain open.
Visual traceability	Figures use ordered flow, text labels, redundant cues, and prose descriptions.	Visual review cannot accept an ADR or qualify performance.
Claims and release	Every local result stays inside a named evidence boundary.	Independent review and all external release gates remain unsatisfied.

The review exposes three high-level conclusions. First, low overhead depends on prepared context and strict ownership, not weaker checks. Second, every retry and recovery path needs exact identity. Third, finite counts need byte and allocator bounds.

16 Evidence and verification plan

The publication source and PDF have a narrow verification contract:

1. Convert each generated light SVG to a vector PDF in a disposable build directory.
2. Build the LaTeX source with fixed time and timezone inputs.
3. Reject LaTeX warnings, overfull boxes, underfull boxes, and undefined references.
4. Require embedded fonts and stable A4 page geometry.
5. Embed and verify a domain-separated SHA-256 commitment over the report, style, and nine SVG sources.
6. Compare the rebuilt PDF with the committed PDF on the same toolchain.
7. Render every page to PNG and inspect layout, labels, equations, tables, and figures.

This contract checks artifact integrity and presentation. It does not validate the protocol or the scientific and safety claims that NCP excludes.

The complete repository gate remains broader. It covers Rust, Python, C, C++, TypeScript, protobuf, schema, profiles, package archives, dependency policy, and candidate baselines. A local pass still cannot satisfy external gates.

The following gates remain separate and **NOT RUN** until exact evidence exists:

- live mTLS, ACL, certificate rotation, and revocation
- two installed and independent non-Rust peers
- fault, soak, duration fuzz, and sanitizer campaigns
- performance qualification
- signatures, SBOM, and provenance
- clean-room reproduction
- all nine exact consumer and extension role qualifications

17 Equation audit and symbol glossary

Table 4 accounts for every numbered equation in this report. The right-hand column states the interpretation and assumptions. It also states the required failure behavior for unavailable or out-of-domain inputs.

Table 4: Equation audit.

Equations	Unit	Meaning, assumptions, and failure behavior
(1)–(4)	Boolean	The receiver evaluates one fixed gate order. It stops at the first false gate and does not inspect later identity-sensitive fields.
(5)–(7)	monotonic ticks	Each local stage has ordered receiver ticks. Negative or cross-incarnation durations are invalid. Contiguity is an accounting boundary.
(8)–(10)	monotonic ticks	Adjacent boundary terms cancel in the telescoping sum. Transport, queue, application, and device time are outside this interval.
(11)–(12)	monotonic ticks	Every selected stage ceiling is finite. B03 must select the ceilings, and platform qualification must test them.
(13)–(15)	operations	Per-stage allocation and full-copy counts add. Zero-copy and zero-allocation are fixed-layout targets, not current measurements.
(16)–(20)	bytes	Finite capacities multiply complete per-unit charges. Fixed overhead remains when every occupied count is zero.
(21)	bytes	Products and partial sums use checked arithmetic in a fixed order. Overflow rejects the profile before allocation.
(22)	local charge	The implementation applies a second finite monotone accounting domain. JavaScript reports an abstract charge unless it controls the complete representation.
(23)–(32)	elements and bytes	Current bounds and deterministic allowable victim selection define the survivor state. Checked reservation establishes the post-admission invariants. Disjoint lifecycle and queue charges then form one portable prepared-memory bound.
(33)–(35)	local charge	Each queue has a finite monotone local charge. Disjoint lifecycle and queue charges add without cross-plane borrowing.
(36)–(38)	operations	One exact namespace has at most one active lifecycle record. A cross-process owner also needs a fenced writer epoch.
(39)–(41)	ordered state transitions	Reservation precedes external work. Result publication and high-water commit precede active-slot release.
(42)–(44)	bytes and Boolean	Retry requires exact coordinate, request, context, and ordinal equality. Any unequal component is a conflict.

Equations	Unit	Meaning, assumptions, and failure behavior
(45)–(48)	ordinal	Zero is the initial high-water sentinel and is never assigned. Checked succession is strictly monotone. Exhaustion seals the epoch instead of wrapping or saturating.
(49)	service count	Session state uses bounded shared owners. The design allocates no thread, socket, or store service per session.
(50)–(55)	position	Position zero is the initial high-water sentinel and is never assigned. Checked succession is strictly monotone. Exhaustion seals the epoch. Assignment commits before later work, so failure creates a visible gap and never permits reuse.
(56)–(59)	bytes and work units	A prepared fixed layout bounds value count and exact frame length. Linear work is a structural count, not elapsed time.
(60)–(66)	requests, bytes, and local charge	Admission leaves one free request slot. Every coefficient is a deployment maximum. Each window reserves portable and local storage before backend work. The active-session capacity lifts both bounds across the deployment.
(67)–(69)	position, digest, and bytes	Only the next exact position executes. Completion advances by checked succession. Response release also requires exact retained request bytes.
(70)–(73)	ticks and Boolean	One receiver clock and body-issued grant define an exclusive deadline. Every currentness, lease, grant, stream, profile, and executor predicate must be true.
(74)	proof relation	NCP does not prove that software admission implies physical effect. The equation does not assert that an effect is impossible.
(75)–(82)	bytes and chunks	Positive chunk capacity and checked offsets produce disjoint half-open ranges whose union is the complete object.
(83)–(87)	bytes and local charge	One activation reserves the complete object plus the larger active or terminal metadata charge. Summation over the bounded activation count lifts both accounting domains across the deployment.
(88)–(91)	bytes and local charge	Auxiliary arenas have independent deployment ceilings. Adding every disjoint owner produces one complete NCP-owned bounded-state envelope.
(92)	graph edge	Consumers depend on NCP through thin adapters. NCP does not import consumer application code.

Table 5: State and result terms.

Term	Exact meaning
Current	The installed receiver-owned state still admits the checked operation
Pending	Capacity and lineage are durable, and external work can be active
Ambiguous	The same operation remains fenced because effect outcome is unknown
Terminal	A definitive result and high-water are durable
Retired	Reexecution is forbidden, while the full retained result can be unavailable
Exact retry	Coordinate, request bytes, context bytes, and ordinal all equal retained values
Software admission	The body executor slot accepted the exact command and evidence transition
Physical effect	A deployment result outside the NCP software proof boundary

18 Conclusion

The proposed architecture obtains low overhead from preparation, exact ownership, one-pass decoding, finite shared stores, and plane isolation. It does not remove authentication, authorization, freshness, safety, or currentness checks.

The equations expose the design accounting used here. Latency is a telescoping sum of named local stages. Memory is the sum of fixed overhead and capacity-scaled charges. Lifecycle mutation is single-flight. Retry is exact equality. Ordinals and positions use checked monotone successors. Queue and extension work reserve complete byte and metadata budgets before retention.

The target remains proposed. Independent B01 review, B02 authorization, B03 allocation, complete implementation, conformance, installed peers, and every external gate still remain.